

WEB DEVELOPMENT – PROJECT

PROJECT TOPIC

Blog REST API with CRUD Operations

FOCUSED AREAS

- Node.js
- Express.js
- REST APIs
- CRUD Operations
- Backend Development
- API Testing

Dear Students,

As part of your Week 06 learning activities, you are required to complete the Project: *Blog REST API with CRUD Operations*.

This project is designed to help you apply your understanding of backend application development, RESTful API design, CRUD operations, and data management using Node.js and Express.js.

You are expected to complete the project independently and submit it before the specified deadline.

1. WHAT IS EXPECTED FROM YOU

Create a Blog REST API that allows users to manage blog posts using CRUD operations:

- Create
- Read
- Update
- Delete

The API should follow RESTful principles and return appropriate HTTP status codes and meaningful JSON responses.

2. FEATURES REQUIRED

2.1 CREATE BLOG POST

Create an API endpoint that allows users to add a new blog post.

Each blog post should contain:

- Title
- Content
- Author Name
- Category
- Created Date

2.2 GET ALL BLOG POSTS

Create an API endpoint to:

- Retrieve all available blog posts.
- Return the blog posts in JSON format.

2.3 GET SINGLE BLOG POST

Create an API endpoint to:

- Retrieve a specific blog post using its ID.
- Return an appropriate response if the requested post does not exist.

2.4 UPDATE BLOG POST

Create an API endpoint to:

- Update an existing blog post.
- Allow modification of the title, content, category, or author.

2.5 DELETE BLOG POST

Create an API endpoint to:

- Delete a blog post using its ID.
- Return appropriate success and error responses.

3. SUGGESTED API ROUTES

- POST /posts → Create a new blog post
- GET /posts → Retrieve all blog posts
- GET /posts/:id → Retrieve a specific blog post
- PUT /posts/:id → Update an existing blog post
- DELETE /posts/:id → Delete a blog post

4. TECHNICAL REQUIREMENTS

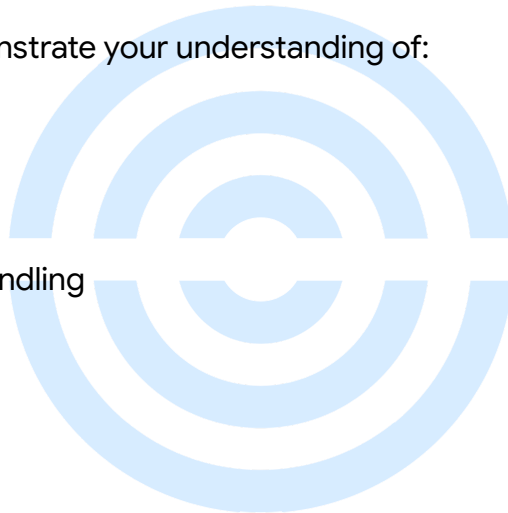
4.1 BACKEND TECHNOLOGIES

- Node.js
- Express.js

4.2 CONCEPTS TO DEMONSTRATE

The project should demonstrate your understanding of:

- REST API Design
- CRUD Operations
- Express Routing
- Middleware
- Request & Response Handling
- Error Handling
- HTTP Status Codes
- JSON Data Handling



5. DATA STORAGE

You may use any one of the following approaches:

- In-memory Array – Basic
- JSON File Storage
- MongoDB – Bonus

6. API TESTING

Test all API endpoints using any one of the following tools:

- Postman
- Thunder Client
- Insomnia

7. PROJECT STRUCTURE

The following project structure is recommended:

project-root/

```
|— routes/  
|— controllers/  
|— middleware/  
|— models/ (optional)  
|— data/  
|— app.js / server.js  
|— package.json
```

8. ADDITIONAL REQUIREMENTS

- Follow RESTful API conventions.
- Implement proper input validation.
- Return meaningful error messages.
- Use appropriate HTTP status codes.
- Organize the code into separate modules.
- Include comments wherever necessary.
- Upload the completed project to GitHub.
- Add a detailed README file containing:
 - Project Overview
 - API Endpoints
 - Technologies Used
 - Installation Steps
 - Testing Instructions

9. SUBMISSION FORMAT

Students must submit the completed project before the specified deadline.

The submission can be made using:

- GitHub Repository Link
- Google Colab Link (if applicable)

Ensure that the submitted link is accessible and that the project repository is properly organized.

10. EVALUATION CRITERIA

The project will be evaluated based on:

- REST API Design
- CRUD Functionality
- Express.js Implementation
- Route Organization
- Error Handling
- Validation Logic
- API Testing Results
- Code Quality and Maintainability
- GitHub Repository Structure
- Documentation Quality

12. BONUS FEATURES

Students may implement the following additional features for bonus consideration:

- MongoDB Integration
- User Authentication using JWT
- Pagination for Blog Posts
- Search and Filter Functionality
- API Documentation using Swagger
- Deployment on Render/Railway

NOTE

Late submissions may not be accepted without prior approval.

Complete the project independently and ensure that your submission demonstrates your understanding of backend development, REST API architecture, and CRUD operations covered during Week 06.

DEADLINE

20th September

Best of Luck!